Python File Handling

A practical guide to open(), read(), write() and append() — written like interview-ready reference notes, with fully commented code.

Every interaction with a file in Python goes through the same three-step pattern: **open** the file to get a file object, **operate** on it (read and/or write), then **close** it to release the resource. Python's built-in `open()` function is the single entry point for all of this, and the *mode* you pass to it decides exactly what you're allowed to do.

1. Opening a File — open()

`open(file, mode='r', encoding=None)` returns a file object. The **mode** string is the most important argument — it controls whether the file is read, written, or appended to, and whether it's treated as text or bytes.

Mode	Meaning	If file exists	If file missing
`'r'`	Read (text)	Opens at start	Raises FileNotFoundError
`'w'`	Write (text)	Truncates (erases!) it	Creates new file
`'a'`	Append (text)	Keeps content, writes at end	Creates new file
`'x'`	Exclusive create	Raises FileExistsError	Creates new file
`'r+'`	Read & write	Opens at start, no truncate	Raises FileNotFoundError
`'b'`	Binary suffix (e.g. 'rb', 'wb')	Works with bytes, not str	—

The safe way: the 'with' statement

Always prefer `with open(...) as f:` over manually calling `open()` / `close()`. The 'with' block is a context manager — it guarantees the file is closed automatically, even if an exception is raised inside the block.

```
# --- Opening a file safely with a context manager ---

# 'with' automatically closes the file when the block ends,
# even if an error happens inside the block.
with open("notes.txt", "r", encoding="utf-8") as f:
    # 'f' is the file object; it only exists inside this block
    contents = f.read()          # read the whole file into a string
    print(contents)

# At this point, outside the 'with' block, the file is ALREADY closed.
# Trying f.read() here would raise: ValueError: I/O operation on closed file.
```

Note: Without 'with', you must call f.close() yourself in a finally block to guarantee the file closes on error. 'with' does this for you -- this is why interviewers expect to see it by default.

2. Reading Files

Python gives you four main ways to pull data out of an open file, each suited to a different situation.

Method	Returns	Best for
`f.read()`	Entire file as one string	Small files you need in memory at once
`f.read(n)`	Next n characters/bytes	Reading in fixed-size chunks
`f.readline()`	Next single line (with \n)	Processing one line at a time manually
`f.readlines()`	List of all lines	When you need list operations (index, slice)
`for line in f:`	Iterator, one line per step	Large files — memory efficient, the Pythonic default

```
# --- Different ways to read a file ---

with open("notes.txt", "r", encoding="utf-8") as f:

    # 1) read() -> loads the ENTIRE file into one string.
    #    Good for small files; risky for huge ones (uses a lot of memory).
    all_text = f.read()

# Re-open because the cursor is now at the end of the file after read()
with open("notes.txt", "r", encoding="utf-8") as f:

    # 2) readline() -> reads just ONE line, including the trailing '\n'.
    #    Calling it again moves forward to the next line.
    first_line = f.readline()
    second_line = f.readline()

with open("notes.txt", "r", encoding="utf-8") as f:

    # 3) readlines() -> returns a LIST, one string per line.
    #    Useful when you need len(), indexing, or slicing.
    all_lines = f.readlines()      # e.g. ['line1\n', 'line2\n', ...]

with open("notes.txt", "r", encoding="utf-8") as f:

    # 4) Iterating directly over the file object -> the most 'Pythonic'
    #    and memory-efficient way. It reads one line at a time internally,
    #    so it works even on files too big to fit in memory.
    for line_number, line in enumerate(f, start=1):
        # .strip() removes the trailing newline character
        print(f"{line_number}: {line.strip()}")
```

3. Writing Files

Mode `'w'` opens a file for writing. **Critical interview point:** if the file already exists, 'w' immediately erases its entire contents the moment it's opened — before you even call write().

```
# --- Writing to a file (mode 'w') ---

# WARNING: if "output.txt" already has content, opening it with 'w'
# wipes that content out immediately, even before write() is called.
with open("output.txt", "w", encoding="utf-8") as f:

    # write() takes a single string and does NOT add a newline for you.
    f.write("First line")
    f.write("Second line")       # this gets glued onto the same line!

    # To separate lines, you must add '\n' explicitly.
    f.write("\nThird line\n")

with open("output.txt", "w", encoding="utf-8") as f:

    # writelines() writes a list of strings back-to-back.
    # Just like write(), it does NOT insert newlines between items --
    # you have to include '\n' in each string yourself.
    lines = ["apple\n", "banana\n", "cherry\n"]
    f.writelines(lines)
```

Note: write() and writelines() never add newlines automatically. This trips people up constantly -- always include \n yourself when you want line breaks.

4. Appending to Files

Mode `'a'` behaves like 'w' in that it will create the file if it doesn't exist, but unlike 'w', it does NOT erase existing content. The file cursor starts at the end, so every write() call adds new data after whatever is already there.

```
# --- Appending to a file (mode 'a') ---

# Run this script multiple times: with 'w' the log would be overwritten
# every run, but with 'a' each run's message is preserved and stacked.
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("Application started\n")

# --- A tiny logging helper built on append mode ---
import datetime

def log_event(message, filename="log.txt"):
    """Append a timestamped message to a log file.

    Using 'a' mode here means we never lose previous log entries,
    which is exactly the behavior you want from a log file.
    """
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(filename, "a", encoding="utf-8") as f:
        f.write(f"[{timestamp}] {message}\n")

log_event("User logged in")
log_event("User uploaded a file")
# log.txt now contains both lines, in order, with timestamps.
```

5. Combined Modes: r+, w+, a+

Adding + to a mode lets you both read and write through the same file object. These are used less often, but interviewers like to check you understand the difference.

Mode	Read?	Write?	Truncates existing content?
r+	Yes	Yes	No — file must already exist
w+	Yes	Yes	Yes — wipes file on open
a+	Yes	Yes	No — writes always go to the end

```
# --- r+ : read and write without truncating ---
# The file must already exist, or FileNotFoundError is raised.
with open("data.txt", "r+", encoding="utf-8") as f:
    first_line = f.readline()     # cursor moves forward as we read
    f.write("Appended after reading\n")   # writes at the CURRENT cursor position,
                                          # which may overwrite existing bytes her
e

# --- seek() and tell(): controlling the cursor manually ---
with open("data.txt", "r+", encoding="utf-8") as f:
    print(f.tell())        # tell() -> current cursor position (in bytes/chars)
    f.seek(0)               # seek(0) -> jump back to the very start of the file
    everything = f.read()   # now reads from the beginning again
```

6. Common Pitfalls (Interview Favorites)

- Using 'w' when you meant 'a' -- silently erases existing data.
- Forgetting encoding="utf-8" -- can cause UnicodeDecodeError on non-English text or on Windows, where the default encoding may not be UTF-8.
- Not using 'with' -- forgetting f.close() can leave file handles open, which on some systems blocks other programs from accessing the file.
- Assuming write() adds newlines -- it doesn't; you must add '\n' yourself.
- Reading a huge file with read() or readlines() -- both load the whole file into memory; iterate with 'for line in f' instead for large files.
- Mixing text and binary mode -- reading an image or PDF with mode 'r' instead of 'rb' raises a UnicodeDecodeError.

7. Full Runnable Example

A complete script tying open, read, write, and append together -- the kind of end-to-end snippet you could paste into a file and run directly, or reproduce on a whiteboard.

```
# file_demo.py
# Complete, runnable demonstration of open/read/write/append in Python.

def write_initial_data(path):
    """Create the file and write its first two lines (mode 'w')"""
    with open(path, "w", encoding="utf-8") as f:
        f.write("Line 1: Hello\n")
        f.write("Line 2: World\n")


def append_more_data(path, extra_lines):
    """Add more lines to the end of the file without erasing it (mode 'a')"""
    with open(path, "a", encoding="utf-8") as f:
        for line in extra_lines:
            f.write(line + "\n")


def read_all_lines(path):
    """Return every line in the file as a list of clean strings"""
    with open(path, "r", encoding="utf-8") as f:
        # strip() removes the trailing '\n' from each line
        return [line.strip() for line in f]


def main():
    path = "demo.txt"

    # Step 1: create the file fresh
    write_initial_data(path)

    # Step 2: append additional lines without wiping step 1's data
    append_more_data(path, ["Line 3: Appended", "Line 4: Also appended"])

    # Step 3: read everything back and print it
    lines = read_all_lines(path)
    for i, line in enumerate(lines, start=1):
        print(f"{i}: {line}")

    # Expected output:
    # 1: Line 1: Hello
    # 2: Line 2: World
    # 3: Line 3: Appended
    # 4: Line 4: Also appended


if __name__ == "__main__":
    main()
```

Quick recap: 'r' reads, 'w' overwrites, 'a' appends, and adding '+' lets a mode both read and write. Always open files with 'with', always specify encoding="utf-8" for text files, and iterate line-by-line for anything large.